Лабораторная работа № 17: Исследование методов создания CLI-приложений.

Цель работы: Сформировать умение анализировать методы создания CLI-приложений.

Задание 1 (Базовый уровень)

Создание простого CLI-приложения с аргументами командной строки

Описание:
Необходимо создать базовое CLI-приложение, которое принимает позиционные аргументы и выводит информацию в терминал. Приложение должно продемонстрировать понимание жизненного цикла CLI: инициализация → логика → завершение с кодом возврата.

Как это должно работать:

1. Приложение запускается командой в терминале
2. При вызове без аргументов выводит справку с описанием использования
3. При вызове с позиционными аргументами выполняет простое действие
4. При успешном выполнении возвращает код 0, при ошибке — код 1

Пример работы:

Вызов справки:

```
text

$ my-cli
Использование: my-cli <команда> [аргументы]
Доступные команды:
  greet <имя>    - поприветствовать пользователя
  info           - вывести информацию о группе
```

Вызов команды приветствия:

```
text

$ my-cli greet "Иван"

Привет, Иван! Добро пожаловать в CLI-приложение группы ББМО-01-23.
```

Вывод информации о группе:

```
text
```

```
$ my-cli info
Группа: ББМО-01-23
Студент: [Ваше ФИО]
Лабораторная работа: №17

Дата: 2024-01-15
```

Обработка ошибки (неизвестная команда):

```
text

$ my-cli unknown
Ошибка: неизвестная команда "unknown"

Для справки используйте: my-cli --help
```

(код возврата 1)

Что нужно сделать для отчёта:

- Сделать скриншот запуска без аргументов (вывод справки)
- Сделать скриншот успешного выполнения команды `greet`
- Сделать скриншот команды `info`
- Сделать скриншот ошибки при неизвестной команде
- Показать код возврата (в Linux/macOS — `echo $?`, в Windows — `echo %ERRORLEVEL%`)

Задание 2 (Средний уровень)

Работа с опциями и флагами (библиотека commander)

Описание:
Необходимо расширить CLI-приложение с использованием библиотеки `commander`. Реализовать поддержку коротких и длинных опций, булевых флагов, опций со значениями, значений по умолчанию и автоматическую генерацию `--help`.

Как это должно работать:

1. Приложение поддерживает несколько подкоманд, каждая со своими опциями
2. Опции могут быть булевыми (`--verbose`, `--force`) и со значениями (`--port 3000`, `--output=file.txt`)
3. Поддерживаются короткие формы (`-p 3000`) и длинные (`--port 3000`)
4. При указании `--help` выводится автоматически сгенерированная справка
5. При указании `--version` выводится версия приложения

Пример работы:

Главная справка:

```
text

$ my-cli --help
Usage: my-cli [options] [command]

CLI-приложение для лабораторной работы №17

Options:
  -V, --version          output the version number
  -v, --verbose          подробный вывод
  -h, --help             display help for command

Commands:
  generate [options]     сгенерировать отчёт
  convert [options]      конвертировать файл
  help [command]         display help for command
```

Справка подкоманды:

```
text

$ my-cli generate --help
Usage: my-cli generate [options]

сгенерировать отчёт

Options:
  -t, --type <type>      тип отчёта (по умолчанию: "html")
  -o, --output <path>    путь для сохранения
  -f, --force            перезаписать существующий файл
  --dry-run              показать что будет сделано без выполнения
  -h, --help             display help for command
```

Выполнение с опциями:

```
text

$ my-cli generate --type pdf --output report.pdf --verbose
[VERBOSE] Запуск генерации отчёта...
[VERBOSE] Тип отчёта: pdf
[VERBOSE] Путь сохранения: report.pdf

Отчёт успешно сгенерирован: report.pdf
```

Режим dry-run:

```
text

$ my-cli generate --type html --dry-run
[DRY-RUN] Будет сгенерирован отчёт типа: html
[DRY-RUN] Файл будет сохранён в: ./output.html

[DRY-RUN] Действия не выполнены (режим проверки)
```

Ошибка валидации:

```
text

$ my-cli generate --type unknown
Ошибка: недопустимый тип отчёта "unknown".

Допустимые значения: html, pdf, json, csv
```

(код возврата 1)

Что нужно сделать для отчёта:

- Сделать скриншот главной справки `--help`
- Сделать скриншот справки подкоманды `generate --help`
- Сделать скриншот выполнения с опциями
- Сделать скриншот режима `--dry-run`
- Сделать скриншот ошибки валидации типа
- Сделать скриншот версии `--version`

Задание 3 (Продвинутый уровень)

Интерактивный режим с библиотекой inquirer

Описание:
Реализовать интерактивный диалог с пользователем через библиотеку `inquirer`.
Приложение должно уметь работать как в интерактивном режиме (задавая вопросы), так и в неинтерактивном (когда все данные переданы через флаги).

Как это должно работать:

1. Если пользователь запускает команду без обязательных флагов — приложение задаёт вопросы
2. Если все данные переданы через флаги — диалог не запускается
3. Поддерживаются разные типы вопросов: `input`, `list`, `checkbox`, `confirm`, `password`
4. Интерактивность отключается флагом `--no-interactive` (для использования в CI/CD)
5. При передаче данных через `stdin` интерактивность отключается автоматически

Пример работы:

Интерактивный режим:

```
text

$ my-cli init
? Введите название проекта: my-project
? Выберите тип проекта: (Use arrow keys)
 ❯ Web-приложение
   CLI-утилита
   Библиотека
   Микросервис
? Выберите дополнительные опции: (Press <space> to select)
 ❯◉ TypeScript
  ◯ ESLint
  ◉ Prettier
  ◯ Jest
? Использовать Git? (Y/n) y
? Введите токен доступа: ********

✓ Проект "my-project" успешно инициализирован!
  Тип: CLI-утилита
  Опции: TypeScript, Prettier

  Git: да
```

Неинтерактивный режим (все данные через флаги):

```
text

$ my-cli init --name my-project --type cli --typescript --prettier --git
✓ Проект "my-project" успешно инициализирован!
  Тип: CLI-утилита
  Опции: TypeScript, Prettier

  Git: да
```

Отключение интерактивности без флагов:

```
text

$ my-cli init --no-interactive

Ошибка: в неинтерактивном режиме необходимо указать --name и --type
```

(код возврата 1)

Передача данных через конвейер:

```
text
```

$ echo "my-project" | my-cli init --no-interactive

Ошибка: недостаточно данных. Укажите --type

Что нужно сделать для отчёта:

- Сделать скриншот полного интерактивного диалога (все типы вопросов)
- Сделать скриншот неинтерактивного режима с флагами
- Сделать скриншот ошибки при `--no-interactive` без данных
- Сделать скриншот передачи данных через `stdin`
- Показать разницу в выводе (интерактив vs автоматизация)

Задание 4 (Сложный уровень)

Форматированный вывод: цвета, спиннеры, прогресс-бары

Описание:
Реализовать профессиональный вывод информации с использованием библиотек `chalk` (цвета), `ora` (спиннеры), `cli-progress` (прогресс-бары). Разделить вывод на `stdout` (данные) и `stderr` (логи, ошибки), соблюдая принцип "Silence is Golden".

Как это должно работать:

1. Цветной вывод (chalk):
    - Успешные операции — зелёным
    - Предупреждения — жёлтым
    - Ошибки — красным
    - Информационные сообщения — синим
    - Цвета отключаются автоматически, если вывод перенаправлен в файл (проверка `process.stdout.isTTY`)
2. Спиннеры (ora):
    - Показываются во время длительных операций
    - Автоматически скрываются при завершении
    - При успехе — зелёная галочка, при ошибке — красный крестик
3. Прогресс-бары:
    - Используются для итеративных процессов (обработка N файлов)
    - Показывают процент выполнения, текущий/общий элемент, оставшееся время
    - Обновляются в одной строке через `\r`
4. Разделение потоков:
    - Результаты работы программы → `stdout`
    - Логи, ошибки, предупреждения → `stderr`
    - При перенаправлении `stdout` в файл — логи всё равно видны в терминале

5. Принцип "Silence is Golden":
    - При успешном выполнении без данных — программа ничего не выводит
    - Только код возврата сообщает об успехе

Пример работы:

Успешное выполнение с выводом:

```
text

$ my-cli process --files a.txt b.txt c.txt
✔ Файлы найдены (3 шт.)
✔ Обработка завершена

Результат сохранён: output.json
```

Выполнение с прогресс-баром:

```
text

$ my-cli process --files *.txt
⠏ Обработка файлов...

[████████████▒▒▒▒▒▒▒▒] 60% | 6/10 | ETA: 00:02
```

Выполнение с ошибками (логи в stderr):

```
text

$ my-cli process --files a.txt missing.txt
✔ Файл a.txt обработан
✖ Файл missing.txt не найден

⚠ Пропущено 1 файл
```

(код возврата 1)

Silence is Golden (всё хорошо, но перенаправляем в файл):

```
text

$ my-cli process --files a.txt > result.json

(в терминале ничего не выводится, данные в result.json)
```

Проверка работы с перенаправлением:

```
text

$ my-cli process --files a.txt > result.json
⚠ Обработано 1 файл из 2 (один отсутствует)
```

```
(в stderr видны логи, в stdout — только данные)
```

Что нужно сделать для отчёта:

- Сделать скриншот цветного вывода (успех/ошибка/предупреждение)
- Сделать скриншот работы спиннера
- Сделать скриншот прогресс-бара
- Сделать скриншот разделения потоков (stdout в файл, stderr в терминал)
- Сделать скриншот демонстрации принципа "Silence is Golden"
- Показать содержимое файла, куда был перенаправлен stdout

Задание 5 (Экспертный уровень)

Полноценное CLI-приложение с архитектурой и упаковкой

Описание:
Создать полноценное CLI-приложение с правильной архитектурой (паттерн "Команда"), обработкой ошибок, тестированием и упаковкой в автономный бинарник.

Как это должно работать:

1. Архитектура проекта:
    - `bin/` — точка входа с шебангом `#!/usr/bin/env node`
    - `commands/` — каждая подкоманда в отдельном файле (`init.js`, `build.js`, `deploy.js`)
    - `services/` — бизнес-логика, изолированная от интерфейса
    - `utils/` — хелперы для форматирования и вывода
    - Модуль `commander` — "тонкий", только маппит флаги в объект опций
2. Обработка ошибок:
    - Слушатели `process.on('uncaughtException')` и `process.on('unhandledRejection')`
    - Понятные сообщения в `stderr` вместо стектрейсов
    - Корректный код возврата (1 при ошибке)
    - Условная логика по `NODE_ENV`: в production — краткие сообщения, в development — детальные трейсы
3. Упаковка и распространение:
    - Секция `bin` в `package.json` (карта "имя команды → путь к файлу")
    - Локальная установка через `npm link` для тестирования
    - Сборка автономного бинарника через `pkg` или `nexe`
    - Бинарник работает без установленного Node.js
4. Тестирование:
    - Юнит-тесты для сервисов (Jest/Mocha), изолированные от ввода-вывода
    - Интеграционные тесты через `child_process.spawn` или `execa`

    - Проверка вывода `--help`, `--version`, кодов возврата
    - Перехват `process.exit` в тестах
5. Команды приложения (пример для группы ББМО-01-23):
    - `my-cli init` — инициализация проекта
    - `my-cli build` — сборка проекта
    - `my-cli test` — запуск тестов
    - `my-cli deploy` — развёртывание (с подтверждением через `--force`)

Пример работы:

Полный цикл работы:

```
text
```

```
$ my-cli init --name my-app --type cli
✔ Проект "my-app" создан
✔ Установлены зависимости
✔ Инициализирован Git-репозиторий
Проект готов к работе!

$ my-cli build
⠋ Сборка проекта...
✔ Сборка завершена за 2.3s
Результат: dist/my-app.js

$ my-cli test
⠋ Запуск тестов...
✔ Пройдено: 15/15 тестов
✔ Покрытие: 87%

$ my-cli deploy --env production
⚠ Вы собираетесь развернуть проект в production!
Продолжить? (y/N) y
⠋ Развёртывание...
✔ Развёрнуто успешно

URL: https://my-app.example.com
```

Работа автономного бинарника (без Node.js):

```
text
```

```
$ ./my-cli-linux --version
1.0.0

$ ./my-cli-linux init --name test

✔ Проект "test" создан
```

Обработка неотловленной ошибки:

```
text

$ my-cli build
✖ Ошибка: файл конфигурации не найден
  Укажите путь через --config

  Подробности: config.json отсутствует в корне проекта
```

(код возврата 1, без стектрейса)

Условная логика по NODE_ENV:

```
text

$ NODE_ENV=development my-cli build
✖ Ошибка: ENOENT: no such file or directory, open 'config.json'
    at Object.openSync (fs.js:498:3)
    at readConfig (/app/services/config.js:15:8)
    ...

$ NODE_ENV=production my-cli build

✖ Ошибка: файл config.json не найден
```

(в development — стектрейс, в production — краткое сообщение)

Запуск тестов:

```
text

$ npm test
PASS  tests/services/build.test.js
PASS  tests/integration/cli.test.js
  ✓ --help выводит справку
  ✓ --version выводит версию
  ✓ неизвестная команда возвращает код 1
  ✓ build с --dry-run не изменяет файлы

Tests: 15 passed, 15 total
```

Что нужно сделать для отчёта:

- Сделать скриншот структуры проекта (дерево файлов)
- Сделать скриншот работы команды `init`
- Сделать скриншот работы команды `build`
- Сделать скриншот работы команды `test`
- Сделать скриншот работы команды `deploy` с подтверждением
- Сделать скриншот работы автономного бинарника (`.exe`/`.bin`)
- Сделать скриншот обработки неотловленной ошибки

- Сделать скриншот разницы вывода при `NODE_ENV=development` и `NODE_ENV=production`
- Сделать скриншот результатов тестирования

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request